

Tópico de Aplicaciones Web

Practica 3 Capa de dominio con genericos y patrones

Unidad I | Plan 2023

Documento de entrega

P03_00000240694

Alumno: Emmanuel Riveros

Matrícula: 00000240694

Que se hizo

En esta práctica completé la capa de dominio de un sistema sencillo de préstamos de biblioteca. La idea principal fue que la regla importante, no prestar un ejemplar que ya está fuera, viviera en el Service. La forma de guardar los datos quedó detrás de un contrato de Repository, así que el Service no depende directamente del Map en memoria.

También dejé funcionando el proyecto base, agregué las dos pruebas que pide la actividad y contesté las preguntas de reflexión con mis propias palabras. El archivo main.ts se mantuvo como venía, porque es el que comprueba el resultado.

Resumen de la solución

Parte	Lo que quedó funcionando
Repository	Contrato genérico con findById, findAll, save y delete, todos con Promise.
PrestamoRepository	Extiende el contrato genérico y agrega findByLibro.
Infraestructura	Map<string, Prestamo> usando el folio como llave.
DTO	Entrada derivada con Omit y salida sin costoReposicion.
Service	Revisa ejemplares no devueltos y lanza el error de dominio si hay choque.
Pruebas	Camino feliz y ejemplar duplicado.

Pasos realizados

1 Preparar el proyecto

Primero instalé las dependencias del proyecto y ejecuté el chequeo de TypeScript. Al inicio los errores venían de los archivos que tenían los TODO; después de completar la solución, el chequeo terminó sin errores.

```
npm install
npm run typecheck
```

No cambié main.ts. Ese archivo sigue armando la aplicación con el repositorio en memoria y ejecuta los escenarios de prueba que ya estaban preparados.

2 Declarar el repositorio genérico

La interfaz Repository quedó con dos tipos: T representa la entidad y ID representa su llave. ID tiene string como valor por defecto. Como todos los métodos regresan promesas, la misma interfaz también serviría si más adelante los datos vinieran de una base de datos.

```
export interface Repository<T, ID = string> {
  findById(id: ID): Promise<T | null>;
  findAll(): Promise<T[]>;
  save(entidad: T): Promise<T>;
  delete(id: ID): Promise<void>;
}
```

Después PrestamoRepository extendió Repository<Prestamo> y agregó findByLibro(libroId). Así se reutilizan las operaciones comunes sin perder la consulta específica que necesita un préstamo.

3 Implementar el repositorio en memoria

La implementación usa el Map que ya venía declarado. La llave es el folio del préstamo y el valor es la entidad completa. En `findById` usé `?? null` para cumplir exactamente con la indicación de devolver `null` cuando no hay coincidencia. `findByLibro` filtra los valores del Map por `libroId`.

```
async findById(id: string): Promise<Prestamo | null> {
  return this.datos.get(id) ?? null;
}

async findByLibro(libroId: string): Promise<Prestamo[]> {
  return [...this.datos.values()]
    .filter((prestamo) => prestamo.libroId === libroId);
}
```

4 Declarar los DTO

Para crear un préstamo, el cliente solo puede mandar `libroId`, `socioid` y `ejemplares`. El folio lo genera el sistema, la fecha también, el estado empieza en `activo` y `costoReposicion` es un dato interno. Por eso el DTO de entrada se derivó desde `Prestamo` con `Omit`, en lugar de copiar los campos a mano.

```
export type CrearPrestamoDto = Omit<
  Prestamo,
  'folio' | 'creadoEn' | 'estado' | 'costoReposicion'
>;
```

El DTO de salida sí lleva la información pública del préstamo, pero no `costoReposicion`. Además, `creadoEn` se convierte a texto ISO para que la respuesta sea fácil de enviar como JSON.

La función `aResponseDto` construye un objeto nuevo y copia el arreglo de ejemplares. Así el dato interno se queda dentro de la entidad y no se filtra hacia la respuesta.

5 Escribir el Service con la regla de negocio

El constructor recibe `PrestamoRepository`, no `InMemoryPrestamoRepository`. Esto deja al `Service` dependiendo del contrato que necesita. Para crear, primero consulto los préstamos del libro, junto los ejemplares de todos los préstamos que todavía no están devueltos y busco si alguno de los nuevos coincide.

```
const ejemplaresFuera = prestamosDelLibro
  .filter((prestamo) => prestamo.estado !== 'devuelto')
  .flatMap((prestamo) => prestamo.ejemplares);

const repetido = dto.ejemplares.find((ejemplar) =>
  ejemplaresFuera.includes(ejemplar),
);
```

Si encuentro una coincidencia, lanzo `EjemplarPrestadoError` con el número del ejemplar. Si no hay choque, creo el folio, pongo la fecha actual, marco el préstamo como `activo`, dejo `costoReposicion` en cero y lo guardo con el repositorio. Los préstamos devueltos no bloquean ejemplares porque se filtran antes de comparar.

`listarPorLibro` solo delega en el repositorio. La regla se concentra en crear, que es donde realmente hace falta validar el préstamo.

6 Verificar el resultado

La verificación completa quedó así:

Comando o escenario	Resultado
<code>npm run typecheck</code>	Terminó sin errores de TypeScript.
<code>npm test</code>	Pasaron 2 pruebas: camino feliz y ejemplar duplicado.

Comando o escenario	Resultado
Escenario 1	Se creó el préstamo P-101 con los ejemplares 14 y 15.
Escenario 2	Falló correctamente con El ejemplar 15 ya esta prestado.
Escenario 3	La consulta de LIB-0417 devolvió un solo préstamo.
Escenario 4	costoReposicion presente: false.

```
npm run typecheck
npm test
npm run dev
```

El resultado confirma que la regla está en el dominio y que el DTO evita exponer el campo interno. También se revisó el Service y no tiene ningún import de InMemory; solo main.ts conoce esa implementación concreta.

7 Pruebas agregadas

Agregué las dos pruebas del servicio que pide la práctica en src/servicios/prestamo.service.test.ts. Las dos crean el servicio con InMemoryPrestamoRepository, pero el constructor del Service sigue tipado con la interfaz PrestamoRepository.

Prueba	Qué comprueba
Camino feliz	Crea un préstamo, revisa libro, socio, ejemplares, estado activo y costoReposicion interno.
Ejemplar duplicado	Crea primero el ejemplar 15 y comprueba que el segundo préstamo lanza EjemplarPrestadoError con el ejemplar correcto.

Respuestas de reflexión

1 Hizo falta una base de datos real

No hizo falta. El Map en memoria alcanzó para guardar y consultar los préstamos mientras probaba la regla. Para mí, esto deja claro que Repository sirve para separar la lógica de negocio de la forma en que se guardan los datos. El Service solo necesita consultar y guardar mediante el contrato; no necesita saber si abajo hay un Map, PostgreSQL u otra tecnología.

2 Qué pasaría usando la clase concreta

Si el Service recibiera InMemoryPrestamoRepository, quedaría amarrado a esa implementación. Si después cambiara la forma de guardar los datos, tendría que modificar el Service aunque la regla de negocio siguiera igual. También sería menos cómodo probarlo con un repositorio falso. Al recibir PrestamoRepository, el Service depende solo de las operaciones que necesita.

3 Cuántos archivos cambiarían al usar una base real

En principio cambiaría el archivo de infraestructura donde está InMemoryPrestamoRepository y cambiaría el ensamblaje de main.ts para crear la nueva implementación. El Service, los DTO y la regla de negocio no tendrían que cambiar. Se tocan pocos archivos porque la interfaz funciona como un punto de conexión entre el dominio y la infraestructura.

Archivos principales

Archivo	Función
src/dominio/repository.ts	Contrato genérico reutilizable.
src/dominio/prestamo.repository.ts	Contrato específico de préstamos.
src/infra/in-memory-prestamo.repository.ts	Persistencia temporal con Map.
src/dto/crear-prestamo.dto.ts	Datos permitidos para crear.
src/dto/prestamo-response.dto.ts	Respuesta pública sin costoReposicion.
src/servicios/prestamo.service.ts	Regla de negocio y creación del préstamo.
src/servicios/prestamo.service.test.ts	Las dos pruebas solicitadas.
README.md	Comandos y respuestas de reflexión.

Enlace del repositorio

Repositorio Frontend Web Practicas: [Abrir repositorio en GitHub](#)

El proyecto incluye el README con las respuestas y puede comprobarse con los comandos indicados en esta entrega.